

Deriving Interpreters:

From Big-Step SOS to Abstract Machines

Mahmoud M. Hamido

Department of Computer Science
RPTU Kaiserslautern-Landau

Supervised by:

Prof. Dr. Ralf Hinze
Alexander Dinges, M.Sc.

May 13, 2026



Outline

1. Motivation
2. Background
3. Case Study I: Arithmetic
4. Case Study II: Exceptions
5. Case Study III: STLC
6. Related Work
7. Conclusion

Motivation: I

- > Implementations should faithfully realize language specifications.
- > A mismatch between specification and implementation is a bug.
- > Testing only checks finitely many inputs.
- > Specifications claim behavior for all well-formed programs.
- > Goal: prove semantic agreement by construction.

Motivation: II

- > Practice: textual specification + conformance tests.
- > Limitation: tests cannot prove absence of mismatch.
- > Approach: encode semantics in Agda, then prove correctness.
- > Curry–Howard: propositions as types, proofs as programs.
- > Result: one proof covers all inputs simultaneously.

Thesis

- > Central claim: derive implementations directly from semantics.
- > Focus: derive abstract machines from big-step semantics.
- > Case studies: Arithmetic, Exceptions, and STLC.

Background: Agda

- > Agda is a dependently typed programming language and proof assistant.
- > Types may depend on values, enabling type-level invariants.

```
data Vec (A : Set) : ℕ → Set where
  []      : Vec A zero
  _::__   : A → Vec A n → Vec A (suc n)
```

- > Here the index tracks list length in the type.

Background: Propositions as Types

- > Truth and falsehood are represented as data types.

```
data  $\top$  : Set where  
  tt :  $\top$ 
```

```
data  $\perp$  : Set where
```

```
ex-falso : {P : Set}  $\rightarrow \perp \rightarrow P$   
ex-falso ()
```

- > A well-typed term is simultaneously a program and a proof.

Background: Connectives in Agda

$\neg$: $\text{Set} \rightarrow \text{Set}$
 $\neg P = P \rightarrow \perp$

$\text{data } _ \times _ (P \ Q : \text{Set}) : \text{Set} \text{ where}$
 $_ , _ : P \rightarrow Q \rightarrow P \times Q$

$\text{data } \Sigma (A : \text{Set}) (B : A \rightarrow \text{Set}) : \text{Set} \text{ where}$
 $_ , _ : (x : A) \rightarrow B \ x \rightarrow \Sigma \ A \ B$

$\text{data } _ \sqcup _ (P \ Q : \text{Set}) : \text{Set} \text{ where}$
 $\text{inl} : P \rightarrow P \sqcup Q$
 $\text{inr} : Q \rightarrow P \sqcup Q$

- > Negation is implication into falsehood.
- > Product and sum model conjunction and disjunction.
- > Σ -types model dependent pairs (existentials).

Background: Equality and Naturals

```
data _≡_ {A : Set} (x : A) : A → Set where  
  refl : x ≡ x
```

```
data IN : Set where  
  zero : IN  
  suc  : IN → IN
```

```
_+_ : IN → IN → IN  
zero + m = m  
suc n + m = suc (n + m)
```

- > Equality is indexed and proved by `refl`.
- > Addition is defined structurally on the first argument.

Background: Helper Lemmas (Rewrite)

$n+0 \equiv n : (n : \mathbb{N}) \rightarrow n + \text{zero} \equiv n$

$n+0 \equiv n \text{ zero} = \text{refl}$

$n+0 \equiv n (\text{suc } n) \text{ **rewrite** } n+0 \equiv n \text{ } n = \text{refl}$

$n+sm \equiv n : (n \text{ } m : \mathbb{N}) \rightarrow n + \text{suc } m \equiv \text{suc } (n + m)$

$n+sm \equiv n \text{ zero } m = \text{refl}$

$n+sm \equiv n (\text{suc } n) \text{ } m \text{ **rewrite** } n+sm \equiv n \text{ } n \text{ } m = \text{refl}$

- > **rewrite** substitutes equals in the current goal.
- > Each recursive case reduces to **refl** after rewriting.

Background: Main Theorem Example

```
+commutative : (n m : ℕ) → n + m ≡ m + n
+commutative zero m rewrite n+0≡n m = refl
+commutative (suc n) m
  rewrite +commutative n m = sym (n+sm≡n m n)
```

- > Strategy: structural induction on n .
- > Base case uses $n+0 \equiv n$.
- > Step case uses induction hypothesis and helper lemma $n+sm \equiv n$.

Background: Termination Obligation

```
diverging :  $\perp$   
diverging = diverging
```

- > Without termination, `ex-falso` would make logic inconsistent.
- > Agda accepts only structurally decreasing recursion by default.
- > `{-# TERMINATING #-}` can bypass this, shifting trust to the programmer.

Background: Variables and Semantics

- > De Bruijn indices avoid capture and α -equivalence issues.
- > Contexts track variable types: $\Gamma ::= \emptyset \mid \Gamma, T$.
- > Typing judgment: $\Gamma \vdash e : T$.
- > Four views of semantics are related in this thesis:

Arithmetic: Syntax and Intrinsic Typing

```
data Type : Set where
```

```
  Nat  : Type
```

```
  Bool : Type
```

```
data ⊢_ : Type → Set where
```

```
  \_      : ⊢ T → ⊢ T
```

```
  _⊞_      : ⊢ Nat → ⊢ Nat → ⊢ Nat
```

```
  test     : ⊢ Nat → ⊢ Bool
```

```
  if_then_else_ : ⊢ Bool → ⊢ T → ⊢ T → ⊢ T
```

- > Ill-typed programs are not representable.

Arithmetic: Big-Step Semantics

```
data _ $\Downarrow$ _ :  $\vdash T \rightarrow \models T \rightarrow \text{Set}$  where
  `_      : (v :  $\models T$ )  $\rightarrow$  (` v)  $\Downarrow$  v
  _ $\boxplus$ _ : e1  $\Downarrow$  n1  $\rightarrow$  e2  $\Downarrow$  n2  $\rightarrow$  (e1  $\boxplus$  e2)  $\Downarrow$  (n1 + n2)
  test    : e  $\Downarrow$  n  $\rightarrow$  (test e)  $\Downarrow$  (test' n)
  if-then : e  $\Downarrow$  true  $\rightarrow$  e1  $\Downarrow$  v  $\rightarrow$  (if e then e1 else e2)  $\Downarrow$  v
  if-else : e  $\Downarrow$  false  $\rightarrow$  e2  $\Downarrow$  v  $\rightarrow$  (if e then e1 else e2)  $\Downarrow$  v
```

> Properties proved: determinism and totality.

Arithmetic: Deterministic and Total

$\text{deterministic} : \forall \{e : \vdash T\} \{v_1 \ v_2 : \models T\}$
 $\rightarrow e \Downarrow v_1 \rightarrow e \Downarrow v_2 \rightarrow v_1 \equiv v_2$

$\text{total} : \forall (e : \vdash T) \rightarrow \exists (\lambda v \rightarrow e \Downarrow v)$

- > Determinism proof: induction on evaluation derivations.
- > In conditional cases, contradictory branch choices yield impossible equalities.
- > Totality proof: structural induction on syntax of expressions.
- > Recursive calls return both a value and a derivation tree.

Arithmetic: Reverse Simulation Proof Shape

$\text{simulate}^R : \forall \{T : \text{Type}\} \{e : \vdash T\} \{v : \models T\}$
 $\rightarrow e \rightarrow^* \backslash v \rightarrow e \Downarrow v$

$\text{eq} = \rightarrow^* \text{-confluence } \text{tr}' \text{ tr } \neg \text{step-} \backslash \neg \text{step-} \backslash$

- > Start from totality: obtain v' with big-step proof $e \Downarrow v'$.
- > Use forward simulation to build $\text{tr}' : e \rightarrow^* \backslash v'$.
- > Use confluence on tr' and given run $\text{tr} : e \rightarrow^* \backslash v$.
- > Since both targets are stuck values, confluence yields $v' \equiv v$.
- > Transport the big-step proof across that equality via subst .

Arithmetic: Small-Step and Simulation

$\text{simulate} : \forall \{T : \text{Type}\} \{e : \vdash T\} \{v : \models T\}$
 $\rightarrow e \Downarrow v \rightarrow e \rightarrow^* \backslash v$

$\text{simulate } (\backslash v) = \text{base}$
 $\text{simulate } (p_1 \boxplus p_2) = \dots$
 $\text{simulate } (\text{if-then } pe \ p_1) = \dots$
 $\text{simulate } (\text{if-else } pe \ p_2) = \dots$

Arithmetic: Denotational Semantics

$\llbracket - \rrbracket : \forall \{T\} \rightarrow \vdash T \rightarrow \models T$

$\llbracket v \rrbracket = v$

$\llbracket e_1 \boxplus e_2 \rrbracket = \llbracket e_1 \rrbracket + \llbracket e_2 \rrbracket$

$\llbracket \text{test } e \rrbracket = \text{test}' \llbracket e \rrbracket$

$\llbracket \text{if } e \text{ then } e_1 \text{ else } e_2 \rrbracket \text{ with } \llbracket e \rrbracket$

$\dots \mid \text{true} = \llbracket e_1 \rrbracket$

$\dots \mid \text{false} = \llbracket e_2 \rrbracket$

> Equivalence theorem: $e \Downarrow v \leftrightarrow \llbracket e \rrbracket \equiv v$.

Arithmetic: Abstract Machine Structure

```
data Frame : Type → Type → Set where
  _else_    : ⊢ T → ⊢ T → Frame T Bool
  _⊞₀_      : Hole → ⊢ Nat → Frame Nat Nat
  _⊞₁_      : ⊢ Nat → Hole → Frame Nat Nat
  test      : Hole → Frame Bool Nat

data State (Answer : Type) : Set where
  ⊢_↓_      : Stack Answer T → ⊢ T → State Answer
  ⊢_↑_      : Stack Answer T → ⊢ T → State Answer
```

Arithmetic: Completeness Proof (Machine)

`complete : $\forall$ (e : $\vdash T$) (v : $\models T$) (stack : Stack Answer T)
→ e $\Downarrow$ v → ($\vdash$ stack $\uparrow$ e) $\rightarrow^*$ ($\vdash$ stack $\downarrow$ v)`

- > Proof by induction on the big-step derivation $e \Downarrow v$.
- > Generalized over arbitrary `stack`, not only `[]`, because recursive calls push frames.
- > Each big-step constructor is matched by a short machine trace.
- > Composition uses transitivity of $\rightarrow^*$ to concatenate subtraces.

Arithmetic: Correctness Proof (Machine)

`correct` : $\forall (e : \vdash T) (v : \models T)$
 $\rightarrow (\vdash [] \uparrow e) \rightarrow^* (\vdash [] \downarrow v) \rightarrow e \Downarrow v$

- > Start from totality of big-step to obtain some v' and proof $e \Downarrow v'$.
- > By completeness, build a canonical machine run $\vdash [] \uparrow e \rightarrow^* \vdash [] \downarrow v'$.
- > Compare canonical run with given run $\vdash [] \uparrow e \rightarrow^* \vdash [] \downarrow v$ using confluence.
- > Project state equality to value equality and transport $e \Downarrow v'$ to $e \Downarrow v$.
- > Equality of final states is projected to equality of returned values.

Arithmetic: Example Completeness Case (if)

```
complete (if e then e1 else e2) v stack (if-then p p1) =  
  step step-if  
    (→*-transitive (complete e _ (stack :: (e1 else e2)) p)  
      (step step-then  
        (complete e1 v stack p1)))
```

- > Push branch frame, evaluate guard under extended stack.
- > After guard returns true, take step-then.
- > Continue by induction hypothesis on selected branch.

Exceptions: Syntax Extension

```
data Exception : Set where  
  err : Exception
```

```
data  $\vdash$ _ where  
  ...  
  throw      : Exception  $\rightarrow \vdash$  T  
  try_catch_ :  $\vdash$  T  $\rightarrow$  (Exception  $\rightarrow \vdash$  T)  $\rightarrow \vdash$  T
```

- > Evaluation now yields either a value or an exception.

Exceptions: Big-Step with Results

```
data Result (T : Type) : Set where
  return :  $\models$  T  $\rightarrow$  Result T
  raise   : Exception  $\rightarrow$  Result T
```

```
data  $\Downarrow^x$  :  $\vdash$  T  $\rightarrow$  Result T  $\rightarrow$  Set where
  throw-rule : throw ex  $\Downarrow^x$  raise ex
  try-return  : e  $\Downarrow$  v  $\rightarrow$  (try e catch h)  $\Downarrow$  v
  try-catch   : e  $\Uparrow$  ex  $\rightarrow$  h ex  $\Downarrow^x$  r  $\rightarrow$  (try e catch h)  $\Downarrow^x$  r
```

- > Every construct has success and propagation behavior.
- > try-return: handler is skipped when body succeeds.
- > try-catch: body exception triggers handler evaluation.

Exceptions: Small-Step Bridge

```
resultExpr : ∀ {T : Type} → Result T → ⊢ T
resultExpr (return v) = ` v
resultExpr (raise ex) = throw ex
```

```
simulate : ∀ {T : Type} {e : ⊢ T} {r : Result T}
  → e ↓r r → e →* resultExpr r
```

- > Final expressions are stuck: neither quoted values nor throws step further.
- > This stuckness lemma is required for the reverse direction via confluence.
- > Lifted ξ -try composes body reduction with outer try-context.

Exceptions: Abstract Machine with Unwind Mode

data State (Answer : Type) : **Set** **where**

$\vdash_{\uparrow}$: Stack Answer T $\rightarrow \vdash$ T $\rightarrow$ State Answer

$\vdash_{\downarrow}$: Stack Answer T $\rightarrow \models$ T $\rightarrow$ State Answer

$\vdash_{\prec}$: Stack Answer T $\rightarrow$ Exception $\rightarrow$ State Answer

step-throw : $\vdash$ stack $\uparrow$ throw ex $\rightarrow \vdash$ stack $\prec$ ex

step-try : $\vdash$ stk $\uparrow$ (try e catch h) $\rightarrow \vdash$ stk :: catch h $\uparrow$ e

step-catch-exn : $\vdash$ stack :: catch h $\prec$ ex $\rightarrow \vdash$ stack $\uparrow$ h ex

Exceptions: Completeness Proofs

`complete` : $e \Downarrow v \rightarrow (\vdash \text{stk} \uparrow e) \rightarrow^* (\vdash \text{stk} \downarrow v)$
`complete-exn` : $e \Uparrow \text{ex} \rightarrow (\vdash \text{stk} \uparrow e) \rightarrow^* (\vdash \text{stk} \prec \text{ex})$

- > Two mutually recursive lemmas are required: value case and exception case.
- > `try-catch` connects both directions:
- > `complete` calls `complete-exn` when the body raises.
- > `complete-exn` calls `complete` when evaluating the handler.
- > Base outcomes differ: values end in $\Downarrow$, exceptions end in $\prec$.

Exceptions: Correctness Proofs

`correct` : $(\vdash [] \uparrow e) \rightarrow^* (\vdash [] \downarrow v) \rightarrow e \Downarrow v$
`correct-exn` : $(\vdash [] \uparrow e) \rightarrow^* (\vdash [] \triangleleft ex) \rightarrow e \Uparrow ex$

- > Proof begins with totality: `e` evaluates to either `return v'` or `raise ex'`.
- > In the matching terminal mode, confluence yields equality of terminal payloads.
- > In the mismatching mode ($\downarrow$ vs $\triangleleft$), confluence yields constructor contradiction.
- > This contradiction is discharged with `ex-falso`, giving the desired result.

Exceptions: Correctness Case Analysis

```
correct e v tr with total e
... | return v' , p = ... confluence ...
... | raise ex , p = ... ex-falso ( $\downarrow \neq <$  ...)
```

- > If totality yields `return v'`, canonical completeness run ends in $\downarrow v'$.
- > Confluence with given run to $\downarrow v$ gives $v' = v$.
- > If totality yields `raise ex`, completeness-exn ends in $< ex$.
- > Confluence with a $\downarrow$ -run yields constructor clash, discharged by absurdity.

STLC: Syntax and De Bruijn Membership

```
data _⊢_ (Γ : Context) : Type → Set where
  Nat  : ℕ → Γ ⊢ Nat
  Bool : ℬ → Γ ⊢ Bool
  Var  : T ∈ Γ → Γ ⊢ T
  λ_   : (Γ , T1) ⊢ T2 → Γ ⊢ (T1 ⇒ T2)
  _·_  : Γ ⊢ (T1 ⇒ T2) → Γ ⊢ T1 → Γ ⊢ T2
```

- Context-indexed syntax guarantees scope and typing discipline.

STLC: Closures and Big-Step Application

```
record Closure (A B : Type) : Set where
  constructor ⟨-, -⟩
  field
```

```
    { $\Gamma^c$ } : Context
     $\delta^c$  : Environment  $\Gamma^c$ 
     $e^c$  : ( $\Gamma^c$ , A)  $\vdash$  B
```

```
app :  $\delta \vdash e_0 \Downarrow \langle \delta^c, e^c \rangle \rightarrow \delta \vdash e_1 \Downarrow v_1$ 
       $\rightarrow (\delta^c, v_1) \vdash e^c \Downarrow v \rightarrow \delta \vdash e_0 \cdot e_1 \Downarrow v$ 
```

- > Application first evaluates function position to a closure.
- > Then evaluates the argument and runs the body in captured environment.

STLC: Totality and Termination Note

```
{-# TERMINATING #-}  
total : ∀ (e : Γ ⊢ T) → ∃ (λ v → δ ⊢ e ⇓ v)
```

```
total (Var x) = ...  
total (λ e)   = ...  
total (e1 · e2) with total e1 | total e2  
... | ... = ...
```

- > STLC is strongly normalizing, but this development uses a termination pragma.
- > Reason: recursive call proceeds through closure body, not obviously structural to Agda.
- > So accepted on programmer trust rather than checker's structural criterion.

STLC: Abstract Machine for Application

```
data Frame : Type → Type → Set where
  app1 : Hole →  $\Gamma \vdash A$  → Frame B ( $A \Rightarrow B$ )
  app2 :  $\models (A \Rightarrow B) \rightarrow$  Hole → Frame B A
  app3 : Environment  $\Gamma \rightarrow \models (A \Rightarrow B) \rightarrow \models A \rightarrow$  Hole → Frame B B

step-· : ( $\delta \vdash$  stack  $\uparrow e_1 \cdot e_2$ )  $\rightarrow$  ( $\delta \vdash$  stack  $::$  app1  $\circ e_2 \uparrow e_1$ )
step-·2 : ...  $\rightarrow$  (( $\delta^c$ ,  $v_2$ )  $\vdash$  stack  $::$  app3  $\delta \langle \delta^c, e^c \rangle v_2 \circ \uparrow e^c$ )
step-·3 : ...  $\rightarrow$  ( $\delta' \vdash$  stack  $\downarrow v_3$ )
```

STLC: Proof Status in This Thesis

complete : $(\delta \vdash \text{stack} \uparrow e \rightarrow^* \delta \vdash \text{stack} \downarrow v)$

- > Completed: big-step semantics, denotational semantics, and machine correspondence.
- > Completed: machine completeness/correctness follows arithmetic-style confluence argument.
- > Open: big-step $\leftrightarrow$ small-step proof is not finished in this thesis.
- > Main obstacle: bridge between substitution-based small-step and environment-based big-step.

STLC: Why Small-Step Equivalence Remains Open

$\text{embed} : \forall \{T : \text{Type}\} \rightarrow \models T \rightarrow \emptyset \vdash T$
 $\text{embed} \{T = T_1 \Rightarrow T_2\} \langle \delta^c, e^c \rangle = \lambda ?$

- > To state simulation, closure values must be re-embedded as syntax.
- > The captured environment in closures prevents a direct embedding.
- > Small-step uses substitution; big-step uses environment lookup.
- > Proving these two mechanisms equivalent is left as future work.

Related Work

- Hutton derives machines by CPS transformation and defunctionalization.
- This thesis derives machines from big-step semantics by reifying frames.
- Conceptual correspondence: $\text{frame} \approx \text{defunctionalized continuation}$.
- Hutton and Wright also study exceptions and verified compilation.

Conclusion: Summary

- > A systematic derivation route from semantics to abstract machines.
- > Demonstrated for arithmetic, exceptions, and STLC.
- > Established equivalence links across semantic styles.
- > Proved machine consistency with big-step semantics in each completed case study.

Conclusion: Limitations and Future Work

- > Current route assumes totality and does not model divergence directly.
- > STLC totality currently uses a trusted termination pragma.
- > STLC small-step $\leftrightarrow$ big-step equivalence is open.
- > Future directions:
 - > Coinductive big-step or small-step-first derivation for divergence.
 - > Strengthen STLC termination and equivalence proofs.
 - > Extend to richer effects and language features.